

Quick Path (quick_path.1.0.2603.exe) 仕様書

1. 製品概要

`quick_path` は、ファイルパスやディレクトリパスをリスト形式で管理し、選択したパスに対して迅速にアクション（エクスプローラー起動、ターミナル起動、パスコピー、カスタムコマンド実行）を行うためのコマンドラインインターフェース（CLI）ツールです。

多くの開発プロジェクトやドキュメントディレクトリを抱えるユーザーの作業効率向上を目的としています。

2. 操作方法 (ユーザーマニュアル)

2.1 起動方法

- 1. `quick_path.1.0.2603.exe` を実行します。
- 2. 同一ディレクトリにある `quick_path.ini` を自動的に読み込みます。（プログラム名が `quick_path.1.0.2603.exe` の場合、最初のピリオドより前の `quick_path` がベースとなります）

2.2 メイン画面の操作

キー	アクション
↑ / ↓ (矢印)	リスト内の項目を選択（青背景でハイライト）
Enter	選択した項目がフォルダなら階層に入り、ファイルならエクスプローラーで開く
[..]	(フォルダ内) 階層を一つ戻る、または設定メニューに戻る
[e]	Explore: 選択した項目をエクスプローラーで開く
[t]	Terminal: 選択した項目を Windows Terminal で開く
[c]	Copy: 選択したパスをクリップボードにコピーする
[q]	Quit: アプリケーションを終了する
カスタムキー	<code>main.ini</code> で定義されたカスタムアクションを実行します

3. 設定方法 (main.ini)

設定ファイル `main.ini` でパスのリストとカスタムコマンドを定義します。

3.1 [Paths] セクション

表示するパスを `表示名 = パス` の形式で定義します。

```
[Paths]
ソースディレクトリ = D:\Source\Projects
ドキュメント = C:\Users\user\Documents
```

3.2 [Commands] / [SyncCommands] セクション

独自のショートカットキーと実行コマンドを定義します。 `キー = 表示名, コマンド` の形式で記述します。
`{path}` プレースホルダーを使用すると、現在選択されているパスに置換されます。

- **[Commands] (非同期実行):** コマンドをバックグラウンドで起動します。ツールはすぐに次の入力を受け付けます。
- **[SyncCommands] (同期実行):** コマンドの終了を待ち、その標準出力および標準エラー出力をクリップボードにコピーします。

```
[Commands]
v = VSCode, code {path}
a = Antigravity, antigravity {path}

[SyncCommands]
g = GitStatus, git status
```

- `v` を押すと、選択中のパスが VS Code で開かれます（非同期）。
- `g` を押すと、`git status` が実行され、その結果がクリップボードにコピーされます（同期）。

4. 論理設計 (システム仕様)

4.1 設定読み込みロジック (load_config)

- 実行ファイルのベース名 (`quick_path.1.0.2603.exe` など) をドットで分割した最初の要素 (例: `quick_path`) に基づき、拡張子 `.ini` を付与したファイルを探します。
- `config.optionxform = str` を設定しているため、`[Paths]` セクションなどのキーは大文字小文字が保持された状態で表示されます。
- `utf-8-sig` エンコーディングで読み込むため、BOM付きの日本語INIファイルにも対応しています。

4.2 実行制御ロジック (run_action / 階層移動)

- **Enterキーの挙動:**
 - 選択中の項目がディレクトリの場合、その階層に移動し、内容を表示します。
 - 選択中の項目が `[..]` の場合、一つ前の表示状態（親ディレクトリまたは初期設定メニュー）に戻ります。
 - 選択中の項目がファイルの場合、標準アクションの **Explore (e)** を実行します。
- **ディレクトリ探索初期化:**
 - `os.scandir` を使用してディレクトリ内の項目を一覧化し、ディレクトリを優先してソート表示します。
- **各アクション実行:**

- 選択されたパスがファイルを指している場合はその親ディレクトリ、フォルダを指している場合はそのパス自体をアクションの実行対象（作業ディレクトリ）と見なします。
- **Terminal ([t]):** `start wt -d "{dir_path}"` コマンドにより独立した Windows Terminal プロセスを起動します。
- **非同期コマンド ([Commands]):**
 - `os.system(f'start "" {final_cmd}')` を使用して実行されます。
 - プロセスは独立して動作し、ツール側の操作を妨げません。
- **同期コマンド ([SyncCommands]):**
 - `subprocess.run` を使用して実行され、終了までツールが待機します。
 - 実行結果（標準出力+標準エラー）は PowerShell の `Set-Clipboard` を介してクリップボードに自動的にコピーされます。
 - `{path}` プレースホルダーは、ダブルクォートで囲まれた絶対パスに置換されます。

5. 物理設計 (ファイル構成・ビルド)

5.1 ファイル構成

- `main.py`: プログラム本体
- `main.ini`: ユーザー設定ファイル (実行時は `quick_path.ini` にリネーム可能)
- `assets/ken2tec.ico`: 実行ファイルのアイコン
- `build.ps1`: ビルド用PowerShellスクリプト

5.2 ビルド環境

- **Python**: 3.x
- **環境**: `dev` という名称の Conda 仮想環境でのビルドを想定しています。
- **主要モジュール**: `pyinstaller`, `configparser`, `msvcrt`, `re`, `subprocess`
- **ビルドコマンド**:

```
# build.ps1 を実行
.\build.ps1
```

内部的には PyInstaller を使用し、依存関係を1つの実行ファイル (`--onefile`) にパッケージングします。不要なライブラリ (pandas, numpy等) はサイズ削減のため明示的に除外されます。

6. 注意点

- **Windows Terminal**: `[t]` アクションは Windows Terminal (`wt`) がインストールされている必要があります。
- **パスのクォート**: カスタムコマンド内で `{path}` を使用する場合、内部で自動的にクォート処理が行われます。